

theorem

Vera: Weakest Preconditions for TICL over Rust

From Imp Structural Lemmas to Rust VCGen Rules

Lef Ioannidis

2026

Background: TICL over Imp

TICL Structural Lemmas for Imp

Core idea (from the TICL paper)

For a small imperative language `StImp` with mutable state, the entailment $[t, m \Vdash_L \varphi]_S$ is proved structurally via inference rules on the syntax of t .

Key rules from `imp-lemmas.tex`:

- **Seq**_SAU_L: sequential composition decomposes into bind
- **If**_SAU_L: conditional branches both satisfy the obligation
- **While**_SAG: infinite loop with coinductive invariant $\mathcal{R}$
- **While**_SAU_L: terminating loop with ranking function f

Goal: Lift these to Rust syntax $\rightarrow$ define $\text{wp}(_, _)$ for Vera.

Notation

$\sigma \in \text{State}$ (mutable references, heap)

$[t, \sigma \Vdash_L \varphi]_{\text{Rust}} \triangleq \text{“Rust term } t \text{ from state } \sigma \text{ satisfies } \varphi\text{”}$

$\text{wp}(t, \varphi) \triangleq \text{weakest precondition on } \sigma \text{ s.t. } [t, \sigma \Vdash_L \varphi]_{\text{Rust}}$

Temporal formulas

- $\varphi ::= \text{now } P \mid \text{AG } \varphi \mid \varphi \text{ AU } \varphi' \mid \varphi \wedge \varphi'$
- Sugar: $\text{AF } \varphi' = \top \text{ AU } \varphi'$, $\text{AX } \varphi' = \top \text{ AN } \varphi'$
- done Q : termination postcondition; $\text{now } P$: intermediate state predicate

Inference Rules

Assignment

Rust: $*x = e$

$$\frac{\begin{array}{c} [e, \sigma \Vdash_R \varphi \text{ AU done } \mathcal{R}]_{\text{Rust}} \\ \forall v, \sigma', \mathcal{R} \ v \ \sigma' \rightarrow [\text{skip}, \sigma'[x \mapsto v] \Vdash_L \varphi \text{ AU } \varphi']_{\text{Rust}} \end{array}}{[*x = e, \sigma \Vdash_L \varphi \text{ AU } \varphi']_{\text{Rust}}} \text{ASSIGN}_{\text{RUST}} \text{AU}_L$$

Evaluating e may take steps (function calls, etc.); φ must hold along the path. After e produces v , the store $\sigma'[x \mapsto v]$ must satisfy φ' .

Sequential Composition (let-binding)

Rust: `let x = t; u`

$$\frac{\begin{array}{c} [t, \sigma \Vdash_R \varphi \text{ AU AX done } \mathcal{R}]_{\text{Rust}} \\ \forall x, \sigma', \mathcal{R} \times \sigma' \rightarrow [u[x], \sigma' \Vdash_L \varphi \text{ AU } \varphi']_{\text{Rust}} \end{array}}{[\text{let } x = t; u, \sigma \Vdash_L \varphi \text{ AU } \varphi']_{\text{Rust}}} \text{LET}_{\text{RUST}} \text{AU}_L$$

Direct lift of **BindAU**_L / **SeqS**AU_L from TIDL.

Rust: `if c {t} else {u}`

$$\frac{\begin{array}{l} [c, \sigma \Vdash_R \varphi \text{ AU done} = (b, \sigma')]_{\text{Rust}} \\ \left\{ \begin{array}{ll} [t, \sigma' \Vdash_L \varphi \text{ AU } \varphi']_{\text{Rust}}, & \text{if } b \\ [u, \sigma' \Vdash_L \varphi \text{ AU } \varphi']_{\text{Rust}}, & \text{otherwise} \end{array} \right. \end{array}}{[\text{if } c \{t\} \text{ else } \{u\}, \sigma \Vdash_L \varphi \text{ AU } \varphi']_{\text{Rust}}} \text{IF}_{\text{RUST}}\text{AU}_L$$

Evaluating c may take multiple steps; φ must hold along the path.

While Loop (AU – terminating)

Rust: `while c {t}` **with** invariant $\mathcal{R}$, decreases f

$$\begin{array}{c}
 \mathcal{R} \sigma \\
 \forall \sigma, \mathcal{R} \sigma \rightarrow [c, \sigma \Vdash_R \varphi \text{ AU done} = (b, \sigma')]_{\text{Rust}} \\
 \wedge \left\{ \begin{array}{l} [t, \sigma' \Vdash_L \varphi \text{ AU } \varphi']_{\text{Rust}} \vee \\ [t, \sigma' \Vdash_R \varphi \text{ AU AF done } (\lambda \sigma''. \mathcal{R} \sigma'' \wedge f \sigma'' < f \sigma)]_{\text{Rust}}, \text{ if } b \\ \varphi'(\sigma'), \text{ otherwise} \end{array} \right. \\
 \hline
 [\text{while } c \{t\}, \sigma \Vdash_L \varphi \text{ AU } \varphi']_{\text{Rust}} \quad \text{WHILE}_{\text{RUST}} \text{AU}
 \end{array}$$

Loop (AG – infinite)

Rust: `loop {t}` **with** invariant $\mathcal{R}$ (**no** decreases)

$$\frac{\begin{array}{c} \mathcal{R} \sigma \\ \forall \sigma, \mathcal{R} \sigma \rightarrow \\ [\text{loop } \{t\}, \sigma \Vdash_L \varphi]_{\text{Rust}} \wedge \\ [t, \sigma \Vdash_R \varphi \text{ AU AF done } (\lambda \sigma' \Rightarrow \mathcal{R} \sigma')]_{\text{Rust}} \end{array}}{[\text{loop } \{t\}, \sigma \Vdash_L \text{AG } \varphi]_{\text{Rust}}} \text{LOOP}_{\text{RUST}} \text{AG}$$

Coinductive hypothesis $[\text{loop } \{t\}, \sigma \Vdash_L \varphi]_{\text{Rust}}$ enables proving φ holds at loop re-entry after each iteration.

Async Bind (Lazy Future Creation)

Rust: `let p = async f( $\vec{e}$ ); k p`

$$\frac{\text{No state change (lazy future)} \quad [k \ p, \ \sigma \Vdash_L \ \Phi]_{\text{Rust}}}{[\text{let } p = \text{async } f(\vec{e}); \ k \ p, \ \sigma \Vdash_L \ \Phi]_{\text{Rust}}} \text{ASYNCBIND}_{\text{RUST}}$$

Rust futures are lazy: `async fn()` returns a handle, body runs at `.await`.

Await (AU/AF – terminating callee)

Rust: `let x = p.await; k x`

$$\frac{\begin{array}{c} [P[p], \sigma \Vdash_R \varphi' \text{ AU AX done } \mathcal{R}]_{\text{Rust}} \\ \varphi' \implies \varphi \text{ (transparent)} \\ \forall x, \sigma', \mathcal{R} \ x \ \sigma' \rightarrow [k \ x, \sigma' \Vdash_L \varphi \text{ AU } \varphi'']_{\text{Rust}} \end{array}}{[\text{let } x = p.\text{await}; k \ x, \sigma \Vdash_L \varphi \text{ AU } \varphi'']_{\text{Rust}}} \text{AWAIT}_{\text{RUST}} \text{AU}_L$$

$P[p]$: async process behind future p . Callee's path φ' must imply caller's φ (transparent await).

Await (AG – diverging callee)

Rust: `p.await` where $P[p]$ satisfies $\text{AG } \psi$

$$\frac{[P[p], \sigma \Vdash_L \text{AG } \psi]_{\text{Rust}} \quad \psi \implies \varphi}{[p.\text{await}, \sigma \Vdash_L \text{AG } \varphi]_{\text{Rust}}} \text{AWAIT}_{\text{RUST}} \text{AG}$$

Callee runs forever $\rightarrow$ await is a *divergence point*. Caller's $\text{AG } \varphi$ discharged if $\psi \implies \varphi$.

Weakest Preconditions

$$\begin{aligned} \text{wp}(*x = e, \varphi \text{ AU } \varphi') &= \lambda \sigma. \text{wp}(e, \varphi \text{ AU done } \mathcal{R}) \sigma \\ &\quad \wedge \forall v, \sigma'. \mathcal{R} \ v \ \sigma' \rightarrow \varphi'(\sigma'[x \mapsto v]) \end{aligned}$$

Evaluate e (maintaining φ), then check φ' after the store.

wp: Let-binding (Sequential Composition)

$$\begin{aligned} \text{wp}(\text{let } x = t; u, \varphi \text{ AU } \varphi') &= \lambda \sigma. \text{wp}(t, \varphi \text{ AU } \text{AX done } \mathcal{R}) \sigma \\ &\quad \wedge \forall x, \sigma'. \mathcal{R} \ x \ \sigma' \rightarrow \text{wp}(u[x], \varphi \text{ AU } \varphi') \sigma' \end{aligned}$$

$$\begin{aligned} \text{wp}(\text{if } c \{t\} \text{ else } \{u\}, \varphi \text{ AU } \varphi') &= \lambda \sigma. \\ \text{wp}(c, \varphi \text{ AU done } \mathcal{R}) \sigma \wedge \forall b, \sigma'. \mathcal{R} b \sigma' \rightarrow \\ &\begin{cases} \text{wp}(t, \varphi \text{ AU } \varphi') \sigma', & \text{if } b \\ \text{wp}(u, \varphi \text{ AU } \varphi') \sigma', & \text{otherwise} \end{cases} \end{aligned}$$

wp: While Loop (AU)

$$\text{wp}(\text{while } c \{t\}, \varphi \text{ AU } \varphi') = \lambda \sigma.$$

$$\mathcal{R} \sigma \wedge \forall \sigma, \mathcal{R} \sigma \rightarrow$$

$$\text{wp}(c, \varphi \text{ AU done } \mathcal{R}') \sigma \wedge \forall b, \sigma'. \mathcal{R}' b \sigma' \rightarrow$$

$$\begin{cases} \text{wp}(t, \varphi \text{ AU } \varphi') \sigma' \vee \\ \text{wp}(t, \varphi \text{ AU AF done } (\lambda _ \sigma''. \mathcal{R} \sigma'' \wedge f \sigma'' < f \sigma)) \sigma', & \text{if } b \\ \varphi'(\sigma'), & \text{o.w.} \end{cases}$$

wp: Loop (AG)

$$\begin{aligned} \text{wp}(\text{loop } \{t\}, \text{AG } \varphi) &= \lambda \sigma. \\ &\mathcal{R} \sigma \wedge \forall \sigma, \mathcal{R} \sigma \rightarrow \\ &[\text{loop } \{t\}, \sigma \Vdash_L \varphi]_{\text{Rust}} \wedge \\ &\text{wp}(t, \varphi \text{ AU } \text{AF done } (\lambda \sigma'. \mathcal{R} \sigma')) \sigma \end{aligned}$$

Coinductive: $[\text{loop } \{t\}, \sigma \Vdash_L \varphi]_{\text{Rust}}$ is the coinductive hypothesis.

wp: Function Call – f ensures AF done $\mathcal{R}_f$

f has terminating postcondition

$$\text{wp}(f(e_1, \dots, e_i), \varphi \text{ AU } \varphi') = \lambda \sigma. \forall x, \sigma'. \mathcal{R}_f x \sigma' \rightarrow \text{wp}(k x, \varphi \text{ AU } \varphi') \sigma'$$

where k is the continuation after the call.

f has temporal ensures Ψ : transparent checking

Additionally assert $\Psi \implies \Phi$ where Φ is the caller's obligation.

f ensures AG ψ : cannot prove AF for it

If f has AG ψ (diverges), then $\text{wp}(f(\vec{e}), \text{AF } \varphi')$ is **unprovable** – a diverging call can never reach the AF goal.

wp: Async Bind

$$\text{wp}(\text{let } p = \text{async } f(\vec{e}); k \ p, \Phi) = \lambda \sigma. \text{wp}(k \ p, \Phi) \sigma$$

No-op: lazy future creation does not change state.

wp: Await (AU)

$$\begin{aligned} \text{wp}(\text{let } x = p.\text{await}; k \ x, \ \varphi \text{ AU } \varphi'') &= \lambda \sigma. \ \varphi' \implies \varphi \\ &\quad \wedge \ \forall x, \sigma'. \ \mathcal{R} \ x \ \sigma' \rightarrow \text{wp}(k \ x, \ \varphi \text{ AU } \varphi'') \ \sigma' \end{aligned}$$

where $P[p]$ ensures $\varphi' \text{ AU } \text{AX done } \mathcal{R}$.

wp: Await (AG)

$$\text{wp}(p.\text{await}, \text{AG } \varphi) = \lambda \sigma. \psi \implies \varphi$$

where $P[p]$ ensures $\text{AG } \psi$.

Callee diverges $\rightarrow$ continuation is dead code $\rightarrow$ no further wp.

Summary

Complete Rule Set

Rust Construct	Rule	Section
<code>*x = e</code>	State update, check φ	Inference
<code>let x = t; u</code>	BindAU _L	Inference
<code>if c {t} else {u}</code>	Case split (effectful c)	Inference
<code>while c {t} + decreases</code>	IterAU _L	Inference
<code>loop {t} (no exit)</code>	IterAG (coinductive)	Inference
<code>let p = async f($\vec{e}$); k</code>	No-op (lazy)	Inference
<code>p.await</code> (AU callee)	Bind + transparent	Inference
<code>p.await</code> (AG callee)	Divergence	Inference
<code>f($\vec{e}$)</code> (AF ensures)	Bind + assume $\mathcal{R}_f$	WP only
<code>f($\vec{e}$)</code> (AG ensures)	Transparent $\Psi \implies \Phi$	WP only
<code>f($\vec{e}$)</code> (AG + AF goal)	Unprovable	WP only

Function calls are contract-dependent $\rightarrow$ only in wp, not inference rules.